



224266499

ML MKHATSHWA

Semester Reflection: Moving from Legacy Java EE to Modern Spring Boot

Introduction: A Shift in Perspective

While most of my teammates spent the semester building out specific React components for our car wash management system, my role was a bit more behind-the-scenes. As the lead backend engineer and system integrator, I was responsible for the "engine" of the project—architecting the Spring Boot ecosystem and making sure it talked to our frontend via Axios.

The biggest challenge wasn't just learning new code; it was unlearning the old ways. Moving from the monolithic, heavy Java EE 7 environment we were taught in previous modules to a modern, API-driven architecture was a total paradigm shift. I had to ditch server-side rendering and stateful beans in favor of stateless microservices and JSON contracts. It was a steep learning curve, but it completely changed how I think about software.

Moving to a Headless, RESTful Backend

In our earlier modules, we built monoliths. The server did everything: it handled the logic, the database, and even rendered the UI using JSPs and Servlets. The backend and frontend were basically glued together. For this project, I had to break that habit and build a "headless" Spring Boot backend that existed only to process data.

I spent most of my time designing `@RestController` classes for the Witbank branch dashboard. This was a huge change because I wasn't returning HTML pages anymore; I was returning data. I learned how the Jackson library works under the hood to turn Java objects into JSON payloads automatically. When the React side needed a daily revenue report or a new booking, I didn't have to write complex parsing logic—Spring just handled the transformation.

I also got much better at using HTTP methods correctly. Instead of just using `doGet` or `doPost` for everything, I structured our API with `@GetMapping`, `@PostMapping`, and `@PutMapping`. Using `ResponseEntity` was another win, as it let me send back specific status codes (like 201 for a new booking or 401 if a login failed). This gave the frontend the info it needed to show proper error messages or success alerts.

The Integration Struggle: Axios and CORS

The hardest part of the project was actually getting the two halves of the app to talk to each other. Because we weren't in a monolith anymore, the React app was running on one port (3000) while Spring Boot was on another (8080).

We used Axios for our API calls, and I had to learn how it handles Promises and JSON payloads to make sure my backend could receive them. The "fun" started when we hit our first CORS (Cross-

Origin Resource Sharing) error. It was a massive headache at first, but it taught me a lot about browser security. I learned that browsers block these requests by default to protect users. To fix it, I had to configure Spring Boot to allow requests from our React origin. Solving that felt like a major milestone in understanding how distributed systems actually work in the real world.

Simplifying Data with Spring Data JPA

Coming from a Java EE background, I was used to the "XML hell" of `persistence.xml` and `web.xml`. Setting up a database used to be an exhausting chore. Spring Boot's "convention over configuration" was a life-saver here.

By just adding a few lines to the `application.properties` file, Spring handled the connection pooling and transaction management for me. We used an H2 database for our prototype, which was great because the whole team could run the project without having to install a full MySQL setup locally.

I also implemented Spring Data JPA, which honestly felt like a cheat code. I replaced hundreds of lines of DAO (Data Access Object) boilerplate with simple interfaces. Seeing Spring generate SQL queries just from a method name like `findByStatusAndDate()` was incredible. Combined with Lombok to get rid of getters and setters, it let me build the entire data layer in a fraction of the time it used to take.

Organizing Logic and Separation of Concerns

As the project grew, I had to make sure the code didn't become a mess. Initially, I saw a lot of logic creeping into the Controllers. Remembering what I learned about EJBs, I knew we needed a proper Service layer.

I refactored the backend to use `@Service` classes. This meant the Controllers only handled the "traffic" (HTTP requests), while the actual "thinking" (like calculating revenue or validating a booking) happened in the Service layer. This made the app much easier to maintain. If we ever need to change how we calculate a price, I only have to look in one place instead of digging through API endpoints.

Final Thoughts

This semester was a massive wake-up call. Moving from Java EE 7 to a decoupled Spring Boot and React stack forced me to stop just "writing code" and start "designing systems."

I've realized that modern backend work isn't about making pretty pages; it's about building reliable, secure, and fast APIs. I'm coming away from this project with a much better understanding of CORS, JPA, and the importance of a clean architecture. Java EE gave me the

foundation, but Spring Boot showed me how the industry actually builds things today. I feel much more confident heading into a career as a full-stack dev now.